# Project Brief: Autoscaling LLM — สเกลตาม QPS หรือสเกลตามคิว?

Oct 18, 2026 · @Prapaporn Rattanatamrong

### ภาพรวมโจทย์

โปรเจกต์นี้ตอบคำถามเดียว: **สำหรับ LLM serving การสเกลตามความยาวคิวรักษาคุณภาพบริการได้ดีกว่าการสเกลตามจำนวน request ต่อวินาที (QPS) หรือไม่** ใช้เวลา 2 สัปดาห์ ทำเป็นทีม 2 คน และส่งผลงานเป็นโปสเตอร์สำหรับ CENTRA 10

**ทำไม QPS อาจหลอกเรา:** เว็บทั่วไปแต่ละ request ใช้เวลาใกล้เคียงกัน แต่ LLM ไม่ใช่ ภาระของแต่ละ request ขึ้นกับความยาว prompt (ยิ่งยาว ช่วง prefill ที่โมเดลอ่าน prompt ยิ่งนาน และกิน KV cache หรือหน่วยความจำ GPU มากขึ้น) และความยาวคำตอบ (ยิ่งยาว ยิ่งครอง GPU นาน)

ตัวอย่าง: ตั้งให้สเกลเมื่อเกิน 2 request ต่อวินาทีต่อเครื่อง

- คำถามสั้น 100 token ที่ 2 req/s แทบไม่ทำให้ GPU รู้สึกอะไร ระบบจึง **เพิ่มเครื่องเกินจำเป็น**
- เอกสารยาว 8,000 token ที่ 2 req/s อาจทำให้คิวยาวจนรอหลายวินาที แต่ระบบยังมองว่าปกติ จึง **เพิ่มเครื่องไม่ทัน**

ส่วน metric `vllm:num_requests_waiting` (จำนวน request ที่รอคิวอยู่ใน vLLM) วัดจากอาการโดยตรง ถ้า GPU รับไม่ไหวไม่ว่าด้วยสาเหตุใด คิวจะยาวขึ้นเอง

**สมมติฐาน:** เมื่อ workload มีความยาวปะปนกัน การสเกลตามคิวจะผ่าน SLO ได้มากกว่า โดยใช้ GPU-hours ไม่มากกว่าการสเกลตาม QPS (SLO คือเกณฑ์คุณภาพที่ตั้งล่วงหน้า ในโปรเจกต์นี้คือ TTFT หรือเวลาจนได้ token แรก ต้องไม่เกิน 2 วินาที)

SkyPilot ซึ่งเป็นเครื่องมือ deploy LLM ข้ามคลาวด์และ Kubernetes ใช้การสเกลตาม QPS (เอกสาร SkyServe Autoscaling) ผลของโปรเจกต์นี้จึงบอกได้ว่ากลไกแบบนี้เหมาะกับ LLM แค่ไหน

## สิ่งที่ต้องมีก่อนเริ่ม

**พื้นฐานของทีม**

- เคย deploy งานบน Kubernetes (Deployment, Service, ดู log ด้วย `kubectl`)
- เคยรัน vLLM และยิง request ผ่าน OpenAI-compatible API
- ไม่ต้องเคยใช้ SkyPilot มาก่อน ส่วนที่ 1 จะพาทำ

**ทรัพยากร**

| รายการ | สเปกขั้นต่ำ | หมายเหตุ |
|---|---|---|
| K8s cluster | มี GPU node อย่างน้อย 3 ตัว | ใช้ของคณะ ไม่ต้องเปิดบัญชีคลาวด์ |
| GPU ต่อ node | 1 ตัว ขนาด 24 GB ขึ้นไป เช่น L4 | จำนวน node = จำนวน replica สูงสุด |
| Prometheus | ติดตั้งใน cluster | เก็บ metric จาก vLLM |
| KEDA | ติดตั้งใน cluster | autoscaler ที่อ่าน metric จาก Prometheus ได้ |

| รายการ | สเปกขั้นต่ำ | หมายเหตุ |
|---|---|---|
| Storage | PersistentVolume ประมาณ 50 GB | เก็บไฟล์โมเดลไว้ล่วงหน้า |

**โมเดล:** ใช้โมเดลเปิดขนาดประมาณ 7B เช่น `Qwen/Qwen2.5-7B-Instruct` ไม่ต้องขอสิทธิ์บน Hugging Face และใหญ่พอให้ GPU 1 ตัวถึงจุดอิ่มตัว ถ้าใช้โมเดลเล็กเกินไป GPU จะไม่เคยเต็ม คิวไม่เกิด และการทดลองจะไม่เห็นความต่าง

## ส่วนที่ 1: ทำความรู้จัก SkyPilot (วันที่ 1–3)

เป้าหมายของส่วนนี้คือ deploy vLLM ผ่าน SkyPilot ได้ และเข้าใจว่าแต่ละคำสั่งทำอะไร ทุกขั้นแก้จาก YAML ขั้นก่อนหน้าเพียงเล็กน้อย ถ้าขั้นไหนพัง ปัญหาจะอยู่ที่ส่วนที่เพิ่งเพิ่มเข้าไป

**แผนที่คำสั่ง (เทียบกับ K8s ที่รู้จักแล้ว)**

| คำสั่ง SkyPilot | ทำอะไร | เทียบกับ K8s คร่าว ๆ |
|---|---|---|
| `sky launch` | เปิดเครื่องหนึ่งชุดแล้วรันงาน | สร้าง Pod ตัวเดียว |
| `sky serve up` | เปิดบริการหลาย replica พร้อม load balancer | Deployment + Service + HPA |
| `sky status` / `sky logs` | ดูสถานะ / ดู log | `kubectl get` / `kubectl logs` |
| `sky down` / `sky serve down` | ปิดและคืนทรัพยากร | `kubectl delete` |

**ระวังคำว่า "cluster":** สำหรับ SkyPilot คำนี้หมายถึงเครื่องหนึ่งชุดที่มันเปิดให้ ซึ่งเมื่อใช้ K8s เป็น backend ก็คือ pod ใน K8s cluster ของคณะ ไม่ใช่ K8s cluster ใหม่

**YAML ของ SkyPilot มี 4 ส่วน:** `resources` (ต้องการเครื่องแบบไหน), `setup` (ติดตั้ง รันครั้งเดียว), `run` (รันงานจริง) และ `service` (ใส่เฉพาะตอนใช้ `sky serve`)

### ขั้นที่ 1: ตรวจการเชื่อมต่อ

```
pip install skypilot-nightly[kubernetes]
sky check
```

ต้องเห็น Kubernetes เป็น enabled ถ้าไม่ผ่าน มักเป็นปัญหา kubeconfig (ไฟล์ตั้งค่าการเชื่อมต่อ cluster)

### ขั้นที่ 2: Hello GPU

```
# hello.yaml
resources:
  infra: k8s        # เวอร์ชันเก่าใช้ cloud: kubernetes
  accelerators: L4:1
run: |
  nvidia-smi
```

```
sky launch -c hello hello.yaml
sky status
sky logs hello
sky down hello
```

เป้าหมายคือเรียนรู้วงจร launch → status → logs → down โดยยังไม่ต้องยุ่งกับ vLLM

**ขั้นที่ 3: vLLM เครื่องเดียว**

```
# single.yaml
resources:
  infra: k8s
  accelerators: L4:1
  ports: 8000
envs:
  MODEL_NAME: Qwen/Qwen2.5-7B-Instruct
setup: |
  pip install vllm      # ใช้เวอร์ชันปัจจุบัน  อย่าล็อกเวอร์ชันเก่าตามตัวอย่างในเว็บ
run: |
  vllm serve $MODEL_NAME --port 8000 --max-model-len 8192
```

```
sky launch -c single single.yaml
sky status --endpoint 8000 single
curl http://<endpoint>/v1/models
```

เช็กว่าได้ชื่อโมเดลกลับมา แล้วลองยิง `/v1/chat/completions` ด้วยคำถามสั้น ๆ

**ขั้นที่ 4: `sky serve` แบบ 2 replica คงที่**

เพิ่มส่วน `service` ต่อท้าย `single.yaml` แล้วบันทึกเป็น `serve.yaml`

```
service:
  readiness_probe:
    path: /v1/models
    initial_delay_seconds: 600
  replicas: 2
```

```
sky serve up -n llm serve.yaml
watch -n10 sky serve status llm
```

สถานะจะยังไม่ READY ระหว่างโหลดโมเดล ซึ่งอาจใช้เวลาหลายนาที เป็นเรื่องปกติ ดูความคืบหน้าได้ด้วย `sky serve logs llm 1` เมื่อ READY แล้วจะได้ endpoint เดียวที่กระจายโหลดให้ทั้ง 2 replica

**ขั้นที่ 5: เปิด autoscaling**

เปลี่ยน `replicas: 2` เป็น

```
  replica_policy:
    min_replicas: 1
    max_replicas: 3
    target_qps_per_replica: 2
```

```
sky serve update llm serve.yaml
```

ลองยิงโหลดเบา ๆ แล้วดูว่าจำนวน replica เพิ่มขึ้นหรือไม่ การตั้งค่านี้คือ **แบบ D** ในการทดลอง

**Checkpoint 1:** แคปหน้าจอ `sky serve status` ตอนที่ replica เพิ่มจาก 1 เป็น 2 ขึ้นไป

## ส่วนที่ 2: ตั้งระบบบน K8s (วันที่ 4–6)

ส่วนนี้สร้างแบบ B (สเกลตาม QPS) และแบบ C (สเกลตามคิว) บน cluster เดียวกัน โดยเปลี่ยนแค่ metric ที่ใช้สเกล ตัวแปรอื่นจึงเท่ากันทุกอย่าง เหตุผลที่ไม่เทียบกับ SkyPilot ตรง ๆ คือ load balancer และ cold start ของ SkyPilot ต่างจาก K8s ผลที่ได้จะมีตัวแปรอื่นปนเข้ามา

### ขั้นที่ 1: เก็บโมเดลไว้ล่วงหน้า

ดาวน์โหลดโมเดลลง PersistentVolume ครั้งเดียว แล้วให้ทุก pod mount volume เดียวกัน ถ้าไม่ทำ ทุก replica ใหม่ต้องดาวน์โหลดเองหลายนาที และ cold start (เวลาตั้งแต่สั่งเพิ่มเครื่องจนรับ request ได้) จะบังผลการทดลองทั้งหมด

### ขั้นที่ 2: Deployment ของ vLLM

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: vllm
spec:
  replicas: 1
  selector:
    matchLabels: {app: vllm}
  template:
    metadata:
      labels: {app: vllm}
    spec:
      containers:
      - name: vllm
        image: vllm/vllm-openai:latest
        args: ["--model", "Qwen/Qwen2.5-7B-Instruct", "--max-model-len", "8192"]
        env:
        - {name: HF_HOME, value: /models}
        ports:
        - containerPort: 8000
        resources:
          limits: {nvidia.com/gpu: 1}
        readinessProbe:
          httpGet: {path: /health, port: 8000}
          periodSeconds: 5
        volumeMounts:
        - {name: models, mountPath: /models}
      volumes:
      - name: models
        persistentVolumeClaim: {claimName: model-cache}
```

สร้าง Service ชี้ไปที่ port 8000 แล้วให้ Prometheus เก็บ metric จาก `/metrics` ของทุก pod (ใช้ ServiceMonitor หรือ PodMonitor ถ้าติดตั้ง Prometheus Operator)

### ขั้นที่ 3: เช็กชื่อ metric จริง

```
kubectl port-forward deploy/vllm 8000:8000
curl -s localhost:8000/metrics | grep -E "num_requests|request_success"
```

ชื่อ metric บางตัวเปลี่ยนไปตามเวอร์ชันของ vLLM ให้ใช้ชื่อที่เห็นจากคำสั่งนี้ และจดเวอร์ชัน vLLM ไว้ในรายงาน

**ขั้นที่ 4: แบบ B สเกลตาม QPS**

```
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: vllm-qps
spec:
  scaleTargetRef: {name: vllm}
  minReplicaCount: 1
  maxReplicaCount: 3
  pollingInterval: 15
  advanced:
    horizontalPodAutoscalerConfig:
      behavior:
        scaleDown:
          stabilizationWindowSeconds: 120
  triggers:
  - type: prometheus
    metadata:
      serverAddress: http://prometheus.monitoring:9090
      query: sum(rate(vllm:request_success_total[1m]))
      threshold: "2"
```

KEDA จะหารค่าจาก query ด้วยจำนวน replica แล้วเทียบกับ `threshold` จึงเท่ากับตั้งเป้า 2 request ต่อวินาทีต่อเครื่อง เหมือน `target_qps_per_replica: 2` ของ SkyPilot ข้อจำกัดคือ metric นี้นับ request ที่ **ทำเสร็จแล้ว** ไม่ใช่ที่เข้ามา ถ้าอยากได้อัตราขาเข้าจริง ให้วัดที่ ingress แทน และระบุไว้ในรายงานว่าใช้แบบไหน

`stabilizationWindowSeconds` คือเวลาที่รอก่อนลดเครื่อง กันไม่ให้ระบบเพิ่มลดเครื่องสลับไปมา

**ขั้นที่ 5: แบบ C สเกลตามคิว**

คัดลอก ScaledObject ของแบบ B แล้วเปลี่ยนแค่ชื่อ query และ threshold

```
      query: sum(vllm:num_requests_waiting)
      threshold: "2"
```

เท่ากับตั้งเป้าว่าเฉลี่ยแล้วแต่ละเครื่องมี request รอคิวไม่เกิน 2 ตัว **ใช้ทีละแบบเท่านั้น** ห้ามมี ScaledObject สองตัวชี้ Deployment เดียวกันพร้อมกัน ลบตัวเก่าก่อนสร้างตัวใหม่ทุกครั้ง

**ขั้นที่ 6: จูน threshold**

ก่อนวัดจริง ให้รันแบบ A (replica คงที่ 1 ตัว) แล้วเพิ่มโหลดทีละขั้นจนกว่า TTFT จะเกิน 2 วินาที จดค่า QPS และความยาวคิว ณ จุดนั้น แล้วใช้เป็นจุดตั้งต้นของ threshold ทั้งสองแบบ ทั้งสองแบบต้องจูนอย่างยุติธรรม ไม่จูนแบบหนึ่งละเอียดกว่าอีกแบบ

**Checkpoint 2:** Grafana dashboard ที่แสดงจำนวน replica, `num_requests_waiting` และอัตรา request บนกราฟเดียวกันตามเวลา

## ส่วนที่ 3: การทดลอง (วันที่ 7–10)

การทดลองมี 4 การตั้งค่า × 4 workload × 3 รอบ รวม 48 รอบ รอบละประมาณ 10–12 นาที ใช้เวลารันรวมราว 10 ชั่วโมง เขียน script ให้รันต่อกันอัตโนมัติได้

**การตั้งค่าที่เทียบ**

| แบบ | สเกลตาม | หน้าที่ |
|---|---|---|
| A. Fixed | ไม่สเกล replica คงที่ 1 ตัว | baseline |
| B. QPS | อัตรา request ต่อ replica (KEDA) | ตัวแทนแนวคิดของ SkyPilot |
| C. Queue | `vllm:num_requests_waiting` ต่อ replica (KEDA) | แบบที่เสนอ |
| D. SkyPilot | `sky serve` ตามส่วนที่ 1 ขั้นที่ 5 | ตัวอ้างอิงจากเครื่องมือจริง |

**ตัวแปรที่ต้องล็อกให้เท่ากันทุกแบบ:** โมเดลและเวอร์ชัน vLLM, รุ่น GPU, `max-model-len`, replica ต่ำสุด 1 และสูงสุด 3, เวลารอก่อนลดเครื่อง และเริ่มทุกรอบจาก 1 replica ที่ warm แล้ว

## Workload 4 แบบ

| Workload | ลักษณะ | สิ่งที่ทดสอบ |
|---|---|---|
| 1. Short | input ~100 token, output ~100 token | QPS สะท้อนภาระได้ดีเมื่อทุก request เบาเท่ากัน |
| 2. Long | input ~6,000 token, output ~200 token | QPS สะท้อนภาระได้ดีเมื่อทุก request หนักเท่ากัน |
| 3. Mixed | short 80% ปนกับ long 20% | **ตัวชี้ขาด** request หนักไม่เท่ากัน |
| 4. Bursty | โหลดต่ำ 3 นาที → สูง 4 เท่า 3 นาที → ต่ำ 3 นาที | ความไวในการตอบสนองและการลดเครื่อง |

สร้างโหลดด้วย `vllm bench serve` แบบ random dataset ตัวอย่างสำหรับ workload 1

```
vllm bench serve \
  --backend openai-chat --endpoint /v1/chat/completions \
  --base-url http://<service> --model Qwen/Qwen2.5-7B-Instruct \
  --dataset-name random --random-input-len 100 --random-output-len 100 \
  --request-rate 4 --num-prompts 2400 \
  --save-result --result-filename short_B_run1.json
```

สำหรับ workload 3 ให้รัน 2 คำสั่งพร้อมกัน คำสั่งหนึ่งเป็น short อีกคำสั่งเป็น long โดยตั้ง `--request-rate` ให้ได้สัดส่วน 80:20 สำหรับ workload 4 ให้รันสามช่วงต่อกันด้วย request rate ต่างกัน ชื่อ flag อาจต่างไปตามเวอร์ชัน ให้เช็กด้วย `vllm bench serve --help`

**ข้อสำคัญ:** ใช้ request rate แบบเดียวกันทุกการตั้งค่า และเลือกระดับโหลดที่ทำให้แบบ A ไม่ผ่าน SLO ชัดเจน ถ้าโหลดเบาเกินไปทุกแบบจะผ่านหมดและไม่เห็นความต่าง

**สิ่งที่วัด**

| ตัวชี้วัด | ได้จากไหน | ความหมาย |
|---|---|---|
| TTFT p50 / p99 | ไฟล์ผลของ `vllm bench serve` | ค่ากลาง และค่าของ request ที่ช้าที่สุด 1% |

| ตัวชี้วัด | ได้จากไหน | ความหมาย |
|---|---|---|
| SLO attainment | คำนวณจาก TTFT ราย request | สัดส่วน request ที่ TTFT ≤ 2 วินาที |
| GPU-hours | จำนวน replica × เวลา จาก Prometheus | ตัวแทนต้นทุน |
| เวลาตอบสนอง | กราฟจำนวน replica ตามเวลา | จากโหลดพุ่งจนเครื่องใหม่รับ request ได้ |
| Cold start | log ของ pod | เวลาตั้งแต่ pod เริ่มจน readiness ผ่าน รายงานแยก |

**ผลที่คาดไว้:** workload 1 และ 2 แบบ B กับ C ควรสูสีกัน ความต่างควรเห็นชัดใน workload 3 ถ้าผลออกมาไม่ตรงสมมติฐานก็ไม่เป็นไร ให้วิเคราะห์ว่าเพราะอะไร ซึ่งมีค่าพอ ๆ กัน

**Checkpoint 3:** ผลรอบแรกของ workload 3 ครบทั้ง 4 แบบ ส่งให้อาจารย์ดูก่อนรันที่เหลือ

## ไทม์ไลน์ 2 สัปดาห์

โปรเจกต์มี 4 checkpoint ทุกตัวต้องผ่านก่อนไปขั้นถัดไป ถ้าติดเกินหนึ่งวันให้แจ้งอาจารย์ทันที ไม่ต้องรอถึงวันส่ง

1. **วันที่ 1–3 — ทำความรู้จัก SkyPilot (ส่วนที่ 1)**
    - Checkpoint 1: `sky serve` สเกลจาก 1 เป็น 2 replica ได้
2. **วันที่ 4–6 — ตั้งระบบบน K8s (ส่วนที่ 2)**
    - Checkpoint 2: dashboard แสดง replica, คิว และอัตรา request บนกราฟเดียว
3. **วันที่ 7–10 — รันการทดลอง (ส่วนที่ 3)**
    - Checkpoint 3: ผล workload 3 รอบแรกครบ 4 แบบ ให้อาจารย์ดูก่อนรันที่เหลือ
4. **วันที่ 11–14 — วิเคราะห์ผลและทำโปสเตอร์**
    - Checkpoint 4: ร่างโปสเตอร์ส่งให้อาจารย์ตรวจอย่างน้อย 2 วันก่อนส่งจริง

**การแบ่งงานในทีม 2 คน:** ทั้งสองคนทำส่วนที่ 1 ด้วยกันเพื่อให้เข้าใจภาพเดียวกัน จากนั้นแยกกันดังนี้

| ช่วง | คนที่ 1: ฝั่งระบบ | คนที่ 2: ฝั่งการทดลอง |
|---|---|---|
| วันที่ 1–3 | ส่วนที่ 1 ด้วยกัน | ส่วนที่ 1 ด้วยกัน |
| วันที่ 4–6 | Deployment, Prometheus, KEDA แบบ B และ C | script สร้างโหลด 4 workload และ script เก็บผลจาก Prometheus |
| วันที่ 7–10 | ดูแลระบบระหว่างรัน สลับการตั้งค่า แก้ปัญหา | รันการทดลองตามคิว ตรวจว่าผลครบทุกรอบ |
| วันที่ 11–14 | เขียนส่วนการตั้งค่าในรายงานและ README | วิเคราะห์ผล ทำกราฟ และร่างโปสเตอร์ |

แบบ D (SkyPilot) ใช้การตั้งค่าจากส่วนที่ 1 ขั้นที่ 5 ได้เลย ไม่ต้องมีคนดูแลแยก การรัน 48 รอบต้องเขียน script ให้รันต่อกันอัตโนมัติ ไม่อย่างนั้นสองคนจะไม่ทันในวันที่ 7–10

### งานที่ต้องส่งและเกณฑ์ประเมิน

**งานที่ต้องส่ง**

- ☐ GitHub repository: YAML ทุกไฟล์, script สร้างโหลด, script วิเคราะห์ผล และ README ที่บอกวิธีรันซ้ำได้
- ☐ ไฟล์ผลดิบทั้ง 48 รอบ (JSON จาก `vllm bench serve` และข้อมูลจำนวน replica จาก Prometheus)
- ☐ รายงานสั้น 4–6 หน้า: คำถาม การตั้งค่า ผล และการวิเคราะห์ ระบุเวอร์ชันของ vLLM, SkyPilot และ KEDA
- ☐ โปสเตอร์สำหรับ CENTRA 10 (ภาษาอังกฤษ)

**กราฟที่โปสเตอร์ต้องมีอย่างน้อย**

1. SLO attainment เทียบกับ GPU-hours ของทุกแบบใน workload 3 เป็นกราฟที่ตอบคำถามหลักได้ในภาพเดียว
2. จำนวน replica ตามเวลา ซ้อนกับความยาวคิว ของแบบ B และ C ใน workload 4
3. TTFT p99 ของทุกแบบในทั้ง 4 workload

**เกณฑ์ประเมิน**

| หัวข้อ | น้ำหนัก | สิ่งที่ดู |
|---|---|---|
| การออกแบบการทดลอง | 25% | ล็อกตัวแปรครบ เทียบอย่างยุติธรรม จูน threshold มีเหตุผล |
| ความถูกต้องและการทำซ้ำได้ | 20% | คนอื่นรันตาม README แล้วได้ผลใกล้เคียง มีผล 3 รอบต่อเงื่อนไข |
| การวิเคราะห์ | 30% | อธิบายได้ว่าทำไมผลออกมาแบบนั้น รวมถึงผลที่ไม่ตรงสมมติฐาน |
| การสื่อสาร | 25% | โปสเตอร์ตอบคำถามหลักได้ใน 30 วินาที กราฟอ่านง่าย มีหน่วยครบ |

## ปัญหาที่พบบ่อยและกฎการใช้ทรัพยากร

| อาการ | สาเหตุที่เป็นไปได้ | วิธีแก้ |
|---|---|---|
| `sky serve status` ไม่ขึ้น READY นานหลายนาที | กำลังโหลดโมเดล | ดู `sky serve logs` ถ้ายังโหลดอยู่ให้รอ ปกติ |
| pod ล้มตอนเริ่มเพราะหน่วยความจำ GPU ไม่พอ | context ยาวเกินไป | ลด `--max-model-len` หรือปรับ `--gpu-memory-utilization` |
| ทุกแบบผ่าน SLO หมด ไม่เห็นความต่าง | โหลดเบาเกินไป | เพิ่ม request rate จนแบบ A ไม่ผ่านชัดเจน |
| คิวเป็นศูนย์ตลอดแม้ TTFT สูง | vLLM รับ request เข้าประมวลผลพร้อมกันหมด | ลด `--max-num-seqs` หรือลองใช้ `vllm:num_requests_running` เป็น metric เสริม |
| replica เพิ่มลดสลับไปมา | เวลารอก่อนลดเครื่องสั้นเกินไป | เพิ่ม `stabilizationWindowSeconds` |
| KEDA ไม่สเกลเลย | query ใน Prometheus คืนค่าว่าง | ลองรัน query ใน Prometheus UI ก่อน และเช็กชื่อ metric ตามส่วนที่ 2 ขั้นที่ 3 |

**กฎการใช้ทรัพยากร**

- ทำเสร็จแล้วต้อง `sky down` หรือ `sky serve down` และลบ ScaledObject ทุกครั้ง GPU ที่ลืมคืนจะทำให้ทีมอื่นใช้ไม่ได้
- ห้ามใส่ Hugging Face token หรือ kubeconfig ลงใน repository ใช้ Secret หรือตัวแปรสภาพแวดล้อมแทน
- จองช่วงเวลารันการทดลองใหญ่ (วันที่ 7–10) กับอาจารย์ล่วงหน้า เพื่อไม่ให้ทีมอื่นใช้ GPU ชนกันจนผลเพี้ยน

**แหล่งอ้างอิง**


- vLLM — Deploying with SkyPilot
- SkyPilot — SkyServe Autoscaling
- SkyPilot — Service YAML spec